Trabajo Práctico Especial: IaC



72.20 Redes de Información

Grupo 2

Matias Rossi, 63202
Juan Pablo Birsa, 64500
Federico Viera, 62022

1. Problemática y contexto

The Store es una tienda online en la que un usuario puede recorrer un catálogo de ítems, armar un carrito y confirmar una compra. Está construida como cinco microservicios independientes que se comunican por HTTP. Hoy se levanta con local.sh, un script de bash que crea un clúster kind, compila las imágenes, las carga y aplica un manifiesto YAML con todos los objetos. Los servicios guardan sus datos en memoria, que se pierden al reiniciarse.

El proyecto se despliega con un script que ejecuta pasos, sin describir el resultado esperado ni registrar qué creó. Este tipo de configuraciones son difíciles de replicar, versionar y modificar. Infraestructuras de proyectos a gran escala tienden a crecer de manera rápida y poco controlada, por lo que se pierde registro de cómo está desplegado el proyecto, y cómo cambiar dicha configuración.

Para este tipo de problemas existe la infraestructura como código (IaC). El estado deseado se describe en archivos declarativos y una herramienta, en nuestro caso Terraform, se encarga de alcanzarlo. Así la infraestructura puede modificarse de forma controlada o eliminarse sin dejar recursos sueltos.

2. Diseño de la solución

Nuestra solución es un proyecto de Terraform que levanta la infraestructura necesaria para que la tienda funcione en AWS. El código sigue la estructura recomendada por HashiCorp y se organiza en cinco módulos propios:

- **Red:** VPC en dos zonas de disponibilidad (us-east-1a y us-east-1b), con una subred pública, una privada y una de datos por zona. Incluye el Internet Gateway, un NAT Gateway por zona, las tablas de ruteo y VPC Endpoints de tipo Gateway (S3, DynamoDB) e Interface (SQS). También incluye el Application Load Balancer, creado con Terraform, que dirige las solicitudes a la EC2 donde corre la aplicación.
- **Cómputo:** Una instancia EC2 con Linux por AZ, sin IP pública, ubicada en una de las subredes privadas. En ella se instalan Docker y kind para ejecutar el clúster de Kubernetes. El control plane y los workers de kind corren como contenedores dentro de la misma instancia y comparten sus recursos. Las EC2 utilizan el perfil existente LabInstanceProfile, que contiene LabRole. El tipo de instancia será configurable y deberá respetar los tamaños permitidos por AWS Academy.
- **Imágenes:** Un repositorio ECR por microservicio. Terraform construye las imágenes (linux/amd64) y las publica con el tag definido en la variable app_version. Esta tarea se ejecuta desde el equipo del grupo con las credenciales del usuario del lab; las EC2 descargan las imágenes mediante LabRole.
- **Datos:** RDS MySQL (Catalog), RDS PostgreSQL (Orders) y ElastiCache Redis (Checkout) en las subredes de datos, accesibles solo desde las EC2 de la aplicación, sus security groups permiten conexiones en los puertos correspondientes. RDS utiliza una configuración Multi-AZ con instancia primaria y standby; Redis utiliza una primaria y una réplica en otra zona. Se incluyen una tabla DynamoDB (Cart) y una cola SQS (Orders).

- **Aplicación:** Un Deployment con dos réplicas y un Service por microservicio, desplegados en kind. La UI se expone mediante un Service de tipo NodePort y un mapeo de puertos hacia las EC2, que recibe las solicitudes del ALB. Las NetworkPolicies limitan el tráfico entre pods; se utilizará un complemento de red que las implemente. Las réplicas se ejecutan dentro de una misma EC2 (por AZ), por lo que no ofrecen tolerancia a la caída de esa instancia ni distribución entre zonas de disponibilidad.

El estado de Terraform se guarda en un bucket S3 cifrado, con versionado y bloqueo nativo. El entorno previsto es AWS Academy Learner Lab en us-east-1. Según su documentación, los servicios utilizados están permitidos, con restricciones sobre sus configuraciones. EC2 admite tamaños hasta large; RDS admite MySQL y PostgreSQL, tamaños hasta medium y almacenamiento gp2 de hasta 100 GB, con Enhanced Monitoring desactivado. Se reutilizan LabRole y LabInstanceProfile. Los recursos consumen el presupuesto del lab, por lo que se estimará su costo y se eliminarán con Terraform al finalizar las pruebas.

3. Scope del POC y casos de uso

El POC abarca la red, una EC2 con kind por AZ, el Application Load Balancer, la construcción y publicación de imágenes en ECR, los recursos de datos administrados conectados a los microservicios, los controles de acceso y el estado remoto en S3. Terraform crea y administra los recursos de AWS y los objetos de la aplicación en Kubernetes. La instalación de Docker y la creación del clúster kind se automatizan durante la preparación de las EC2.

EKS queda fuera del alcance principal y podrá incorporarse como una ampliación. También quedan fuera TLS, integración continua, observabilidad, autoescalado y permisos de IAM separados por microservicio.

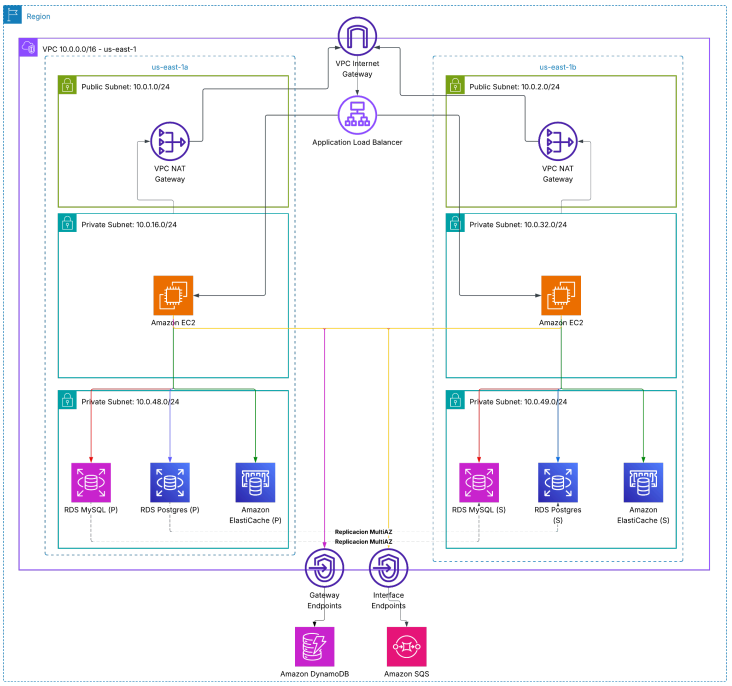
Salvo el primero, cada caso de uso consiste en cambiar un valor en terraform.tfvars y ejecutar plan y apply:

1. **Entorno reproducible y detección de desvíos.** Se crea el entorno completo desde cero y se verifica su funcionamiento. Luego se modifica manualmente, desde la consola de AWS, un recurso administrado por Terraform. Se ejecuta terraform plan para detectar el desvío (drift) y terraform apply para restablecer la configuración declarada. Finalmente, se destruyen primero los recursos de aplicación de ambos clústeres Kind y después la infraestructura AWS, verificando que no queden recursos huérfanos.

2. **Aumento de réplicas para una campaña de ventas.** Se incrementan las réplicas de la UI de 2 a 4 en terraform.tfvars y se ejecutan plan y apply. Kubernetes crea los nuevos pods dentro del clúster kind. La prueba permite observar cómo se modifica el despliegue, aunque la capacidad total sigue limitada por la CPU y la memoria de la EC2. Restaurar el valor anterior vuelve a dejar dos réplicas.
3. **Nueva versión de la aplicación y vuelta atrás.** Se cambia app_version en terraform.tfvars para seleccionar imágenes inmutables previamente publicadas en ECR. Terraform actualiza los Deployments de ambos Kind y Kubernetes realiza el rolling update. Se verifica que la UI continúe respondiendo. Restaurar el valor anterior despliega las imágenes de la versión previa, sin reconstruirlas.

4. **Contención de un microservicio comprometido.** Se agrega Checkout a la lista quarantine. El plan muestra los cambios en las NetworkPolicies que eliminan sus permisos de entrada y salida. Al aplicarlos, Checkout queda aislado. El catálogo y el carrito siguen funcionando; solo la compra queda fuera de servicio hasta vaciar la lista.

4. Diagrama de arquitectura



La infraestructura se distribuye en dos zonas de disponibilidad de us-east-1. Se separa el ingreso público de los recursos internos según el siguiente direccionamiento:

Bloque	Ubicación	Recursos
10.0.0.0/16	VPC	Red del proyecto.
10.0.1.0/24 y 10.0.2.0/24	Subredes públicas, zonas a y b	Application Load Balancer y NAT Gateway de cada AZ.
10.0.16.0/24 y 10.0.32.0/24	Subredes privadas, zonas a y b	EC2 con kind en cada una de las subredes e interfaces del endpoint de SQS.
10.0.48.0/24 y 10.0.49.0/24	Subredes de datos, zonas a y b	RDS MySQL, RDS PostgreSQL y ElastiCache

Bloque	Ubicación
172.20.0.0/16	Red interna de Services de kind
10.244.0.0/16	Red interna de pods de kind

Ruteo y Cómputo

Las subredes públicas salen a Internet por el Internet Gateway. Las privadas lo hacen por el NAT Gateway de su zona, y las subredes de datos conservan únicamente la ruta local de la VPC. La aplicación corre en una EC2 por AZ sin IP pública, ubicada en una subred privada. Dentro de esa instancia, kind ejecuta el control plane y los workers como contenedores. Los pods y los Services utilizan redes internas de Kubernetes y no reciben direcciones de las subredes de AWS. El grupo administra Kubernetes desde las EC2 mediante un acceso autenticado.

Flujo de tráfico

El usuario llega por HTTP al ALB en el puerto 80. El balanceador reenvía la solicitud al puerto 8080 de la EC2, registrado como destino. Un mapeo de puertos de kind la dirige al Service NodePort de la UI, que la entrega a uno de sus pods. La UI llama por HTTP a Catalog, Cart y Checkout a través de sus Services, y Checkout llama a Orders. Catalog se conecta a MySQL (TCP 3306), Orders a PostgreSQL (TCP 5432) y Checkout a Redis (TCP 6379). Cart accede a DynamoDB y Orders pública en SQS por HTTPS, a través de los VPC Endpoints: Gateway para DynamoDB y S3, Interface para SQS. Las imágenes de ECR se descargan desde la EC2 a través del NAT Gateway, y sus capas desde S3 mediante el Gateway Endpoint. Las respuestas al usuario vuelven por la misma conexión del ALB, sin pasar por el NAT.

Controles de acceso

El security group del ALB permite recibir tráfico HTTP desde Internet. El de la EC2 permite recibir solicitudes de la aplicación en el puerto 8080 únicamente desde el security group del ALB. Las bases aceptan conexiones en sus respectivos puertos solo desde el security group de la EC2. Las NetworkPolicies controlan las comunicaciones entre pods y restringen el acceso a los servicios internos según qué microservicios los utilizan. La EC2 utiliza LabRole mediante LabInstanceProfile para acceder a los servicios de AWS que requiere la aplicación. Se reutilizan estos recursos de IAM, sin crear nuevos roles.

5. Alternativas consideradas

- **Pulumi en lugar de Terraform:** Pulumi permite describir la infraestructura en lenguajes de propósito general (Python, TypeScript, Go). Se eligió Terraform porque HCL es declarativo y acotado, el grupo ya lo conocía y dispone de un amplio ecosistema de módulos para AWS.
- **nginx con NLB en lugar de ALB:** La alternativa considerada combina un NLB, que trabaja en capa 4, con un proxy nginx dentro del clúster para resolver el enrutamiento HTTP. Esto agrega un componente que hay que mantener y escalar. Se mantiene el ALB porque la aplicación utiliza HTTP y permite balancear las solicitudes hacia la UI. En esta versión, su destino es el puerto publicado en la EC2, desde donde el tráfico ingresa al clúster kind.